

SimpleShell.c

Functions used:

1. Input()
Reads user input, removes extra spaces and newline characters, and returns the cleaned command string.
2. Input_Parser()
Breaks the user input into individual words and stores them in an array terminated by NULL.
3. Sig_Child()
Handles the SIGCHLD signal. Updates process completion time and prevents creation of zombie processes.
4. print_final_report()
Prints the final report showing each command's PID, completion time, and waiting time. Also calculates the average completion and waiting times.
5. Sig_Int()
Handles Ctrl+C signal. Terminates the scheduler, closes the pipe, and prints the final report before exiting.
6. Run_Command()
Executes normal commands not submitted to the scheduler.
Records start and end time and stores the command details in the history.
7. Launch()
Handles the submit command. Creates a child process for the submitted job, stops it, and sends the job details (PID and name) to the scheduler.
Also handles the history and time commands.
8. main()
Starts the shell, initializes the scheduler using fork and pipe, sets up signal handlers, and manages user input and program exit.

SimpleScheduler.c

Functions used:

1. `time_diff_ms()`
Calculates the difference between two time points in milliseconds.
2. `check_empty()`
Checks if the Ready Queue is empty.
3. `enqueue()`
Adds a new job into the Ready Queue with process ID, name, and arrival time.
4. `dequeue()`
Removes and returns the first job in the Ready Queue.
5. `remove_completed()`
Removes all jobs that have completed execution and rebuilds the queue.
6. `main()`
Implements the round robin scheduling algorithm.
Receives jobs from the shell, resumes and pauses them based on the time slice.
Calculates turnaround time and waiting time for each job.
Prints average completion time when all jobs are finished and exits gracefully.

Link : <https://github.com/Swaraaj333/group-48>

Prerak Tanwar (2024436)

- a) `Input()`
- b) `Run_Command()`
- c) `History()`
- d) `History_exit()`
- e) Changes in Shell , submit command
- f) `Input_Parser()`
- g) `Launch()`
- h) `print_final_report()`

Swaraaj Krishna (2024574)

- a) `time_diff_ms()`
- b) `check_empty()`
- c) `enqueue()`
- d) `dequeue()`
- e) `remove_completed()`
- f) `main()`
- g) `submit`
- i) `time_diff_ms()`

AI Policy

I have referred to AI generated code for this part and understood and wrote it to use in our code.

1)Sigchild function

```
void Sig_Child(int sig) {
    int err_no_save = errno;
    pid_t pid;
    int status;
    struct timeval t_end;

    while((pid = waitpid(-1, &status, WNOHANG)) > 0) {

        gettimeofday(&t_end, NULL);
        for(int i = 0; i < history_counter; i++) {

            if(history[i].pid == pid && history[i].t_finish <= 0.0) {
                double t_finish = (t_end.tv_sec -
history[i].t_start.tv_sec) +
                                (t_end.tv_usec -
history[i].t_start.tv_usec) / 1e6;
                history[i].t_finish = t_finish;
                break;
            }
        }
    }
    errno = err_no_save;
}
```

2)Waiting time and time related info

```
for (int j = 0; j < J_MAX; j++) {
    if (Ready_Queue[j].pid == pid) {
        Ready_Queue[j].completed = 1;
        Ready_Queue[j].t_end = end;
        // this i have handled in shell teport
        double turnaround =
time_diff_ms(Ready_Queue[j].t_arrival, end);
        double wait = turnaround -
(Ready_Queue[j].slice_run * t_slice);

        printf("PID %d has completed.\n", pid);
        printf("Turnaround Time (T_complete): %.2f
ms\n", turnaround);

        printf("Waiting Time (T_wait): %.2f ms\n",
wait);

        fflush(stdout);

        total_turnaround += turnaround;
        completed_jobs++;
        break;
    }
}
```